

DSA STUDY BLUEPRINT SERIES

79. Height of a Binary Tree Count of Nodes

Tree Height Depth Searches and Nodes Count

Section 09 • C++ Core Data Structures & Algorithms

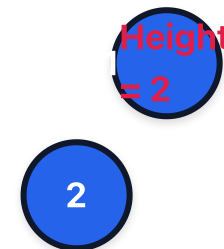
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Height:** Maximum edge distance from root node down to leaf. Formula: $1 + \max(\text{leftHeight}, \text{rightHeight})$.
- **Nodes count:** Recursive count: $1 + \text{leftCount} + \text{rightCount}$.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
int getHeight(TreeNode* root) {  
    if (!root) return 0;  
    return 1 + max(getHeight(root->left), getHeight(root->right));  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(H)$ recursion stack

Key Practices & Gotchas

- **Important:** Null node base cases return 0 height.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace recursive Preorder Traversal visits (Node-Left-Right) for tree `1 (left=2, right=3)`:

Call Stack Level	Active Node Visited	Expected Left Traversal	Expected Right Traversal
1	Root (1)	Recurse on Left child (2)	Recurse on Right child (3)
2	Leaf (2)	Base case (nullptr)	Base case (nullptr)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Iterative traversals:** Standard DFS traversals can run iteratively by simulating recursion calls with Stacks.
- **Level Order BFS:** Level tracking searches use Queues to visit node layers.
- **Related LeetCode Problems:** #102 Binary Tree Level Order Traversal, #94 Binary Tree Inorder Traversal, #236 Lowest Common Ancestor.